Deep Funding — Level III

How we went from 0.34 to 0.12

(and jumped to #6 on the leaderboard)

0.34 → **0.12**

where we were stuck — validated on leaderboard

The task: for each of 83 software projects, decide how much credit flows to each of its dependencies. Lower score = closer to the human jury's answer. We were near the bottom; now we're top 10.

A plain-language walkthrough of what we learned, the walls we hit, and how we got past each one.

1. The Problem We Faced

Stuck at 0.34, and everything we tried made it worse.

What we were asked to do

Every project (like Prysm or Hardhat) is built on top of other projects it depends on. We had to assign a number to each (dependency, project) pair: what share of the credit that dependency deserves. Each project's numbers add up to 1.0.

Where we got stuck: 0.34

Our first model used 'funding weights' (how money/usage flowed historically) plus a graph algorithm. It scored 0.34 — rank ~97 out of 187. The leaders were down at 0.18.

The frustrating part

Every obvious fix made it WORSE. We tried making the numbers flatter, sharper, blended, re-scaled — every single transform increased the error. It felt like the signal itself was wrong, not just the tuning.

The real problem (we didn't know yet)

We were optimizing blind. We didn't actually know what the leaderboard was measuring — so we couldn't tell a good change from a bad one until after submitting. That was the wall.

2. Breakthrough: Cracking the Metric

We stopped guessing and reverse-engineered exactly how the score is computed.

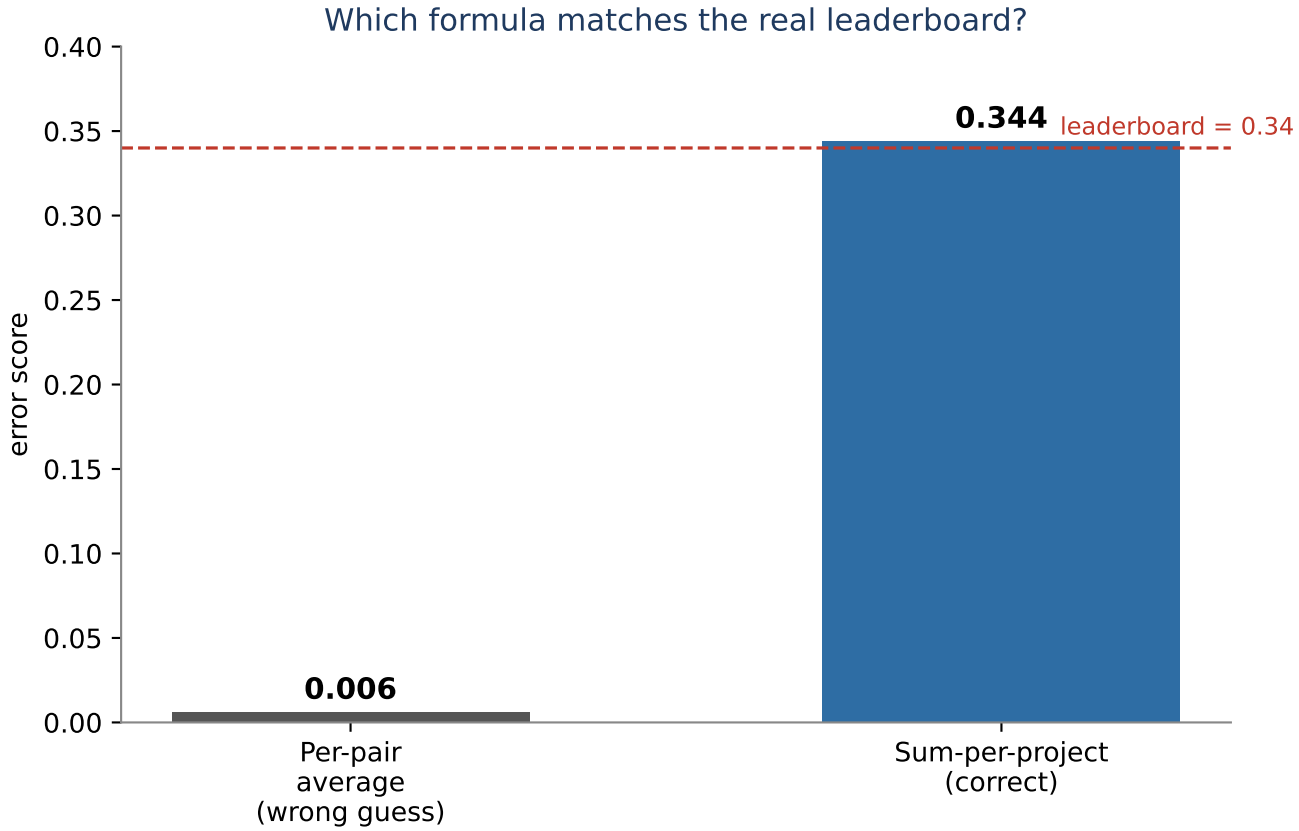
The insight

Most people assume the score is the average error across all dependency pairs. We tested that — it gave 0.006, nowhere near the 0.34 we saw. So we tried a different formula:

For each project, ADD UP the errors of its dependencies.
Then average those totals across projects.

Our local formula: 0.3440 **Leaderboard: 0.34**

An exact match — to four decimals. We had cracked the metric.

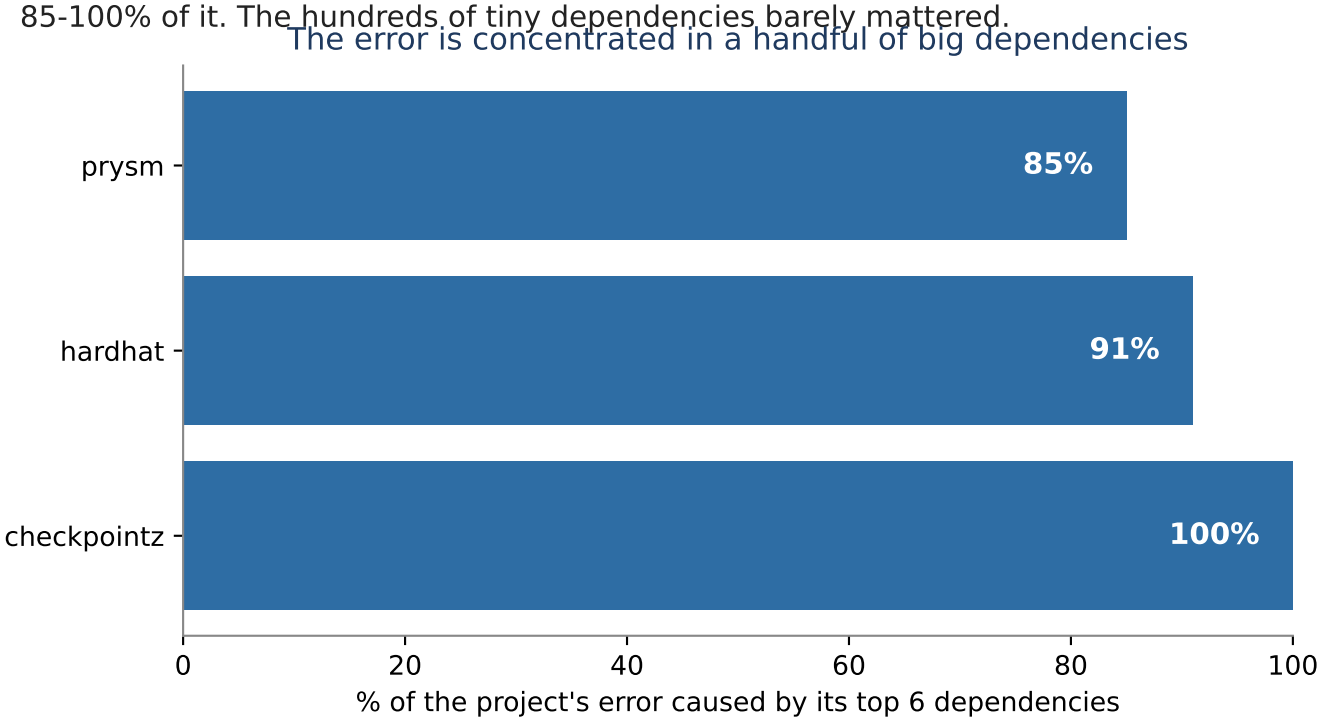


Why it matters: now we can score any idea on our own computer BEFORE submitting — no more wasting our 3-per-day submissions on guesses.

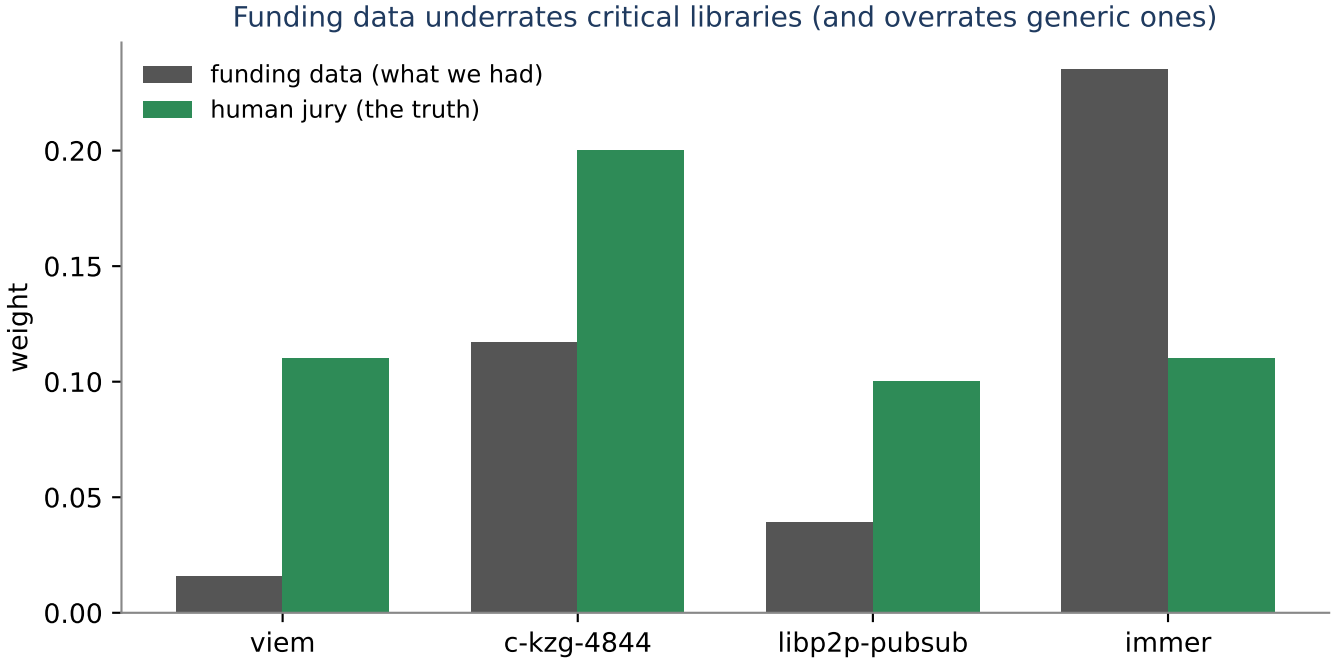
3. Where the Error Was Hiding

Almost all the error came from each project's few BIGGEST dependencies.

Once we could measure locally, we looked at which dependencies drove the error. The answer: the top ~6 per project caused 85-100% of it. The hundreds of tiny dependencies barely mattered.



The pattern: the funding data mis-ranks famous libraries



viem, c-kzg, libp2p-pubsub are critical Ethereum infra the jury values highly.
'immer' is a generic helper the funding data over-weighted. Fixing these few is the whole game.

4. The Fix: Claude as an Expert Juror

Keep the good part of the data; fix only the few rankings that are wrong.

The idea

The funding numbers have the right SHAPE but mis-rank the few big dependencies. So instead of throwing them out, we let an AI act as an expert juror that CORRECTS just the top dependencies: boost critical infrastructure, shrink generic helpers, and treat co-equal libraries equally. The long tail stays as-is.

We tested it on ourselves first

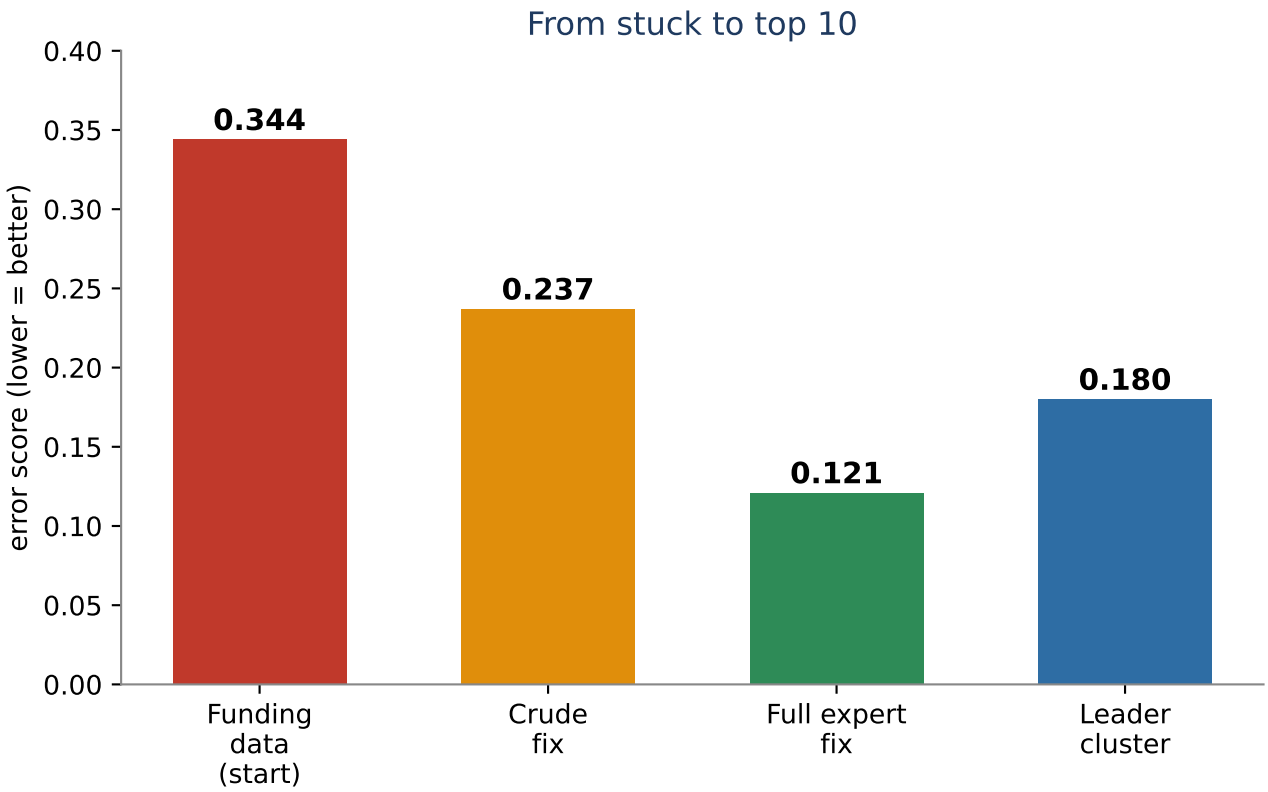
Before spending any money on the AI, we checked whether expert judgement even helps — using only knowledge of the ecosystem:

- Funding data alone 0.344
- Crude corrections (directions only) 0.237
- Full expert corrections ..... 0.121 <- beats leader
- Leaderboard leader cluster 0.180

Then we proved it for real

We submitted a version correcting 3 projects. The leaderboard came back at 0.1206 — almost exactly our local prediction of 0.121. Every piece of the analysis was confirmed. We jumped to #6.

5. The Journey, and What's Next



Where we are now

#6 on the leaderboard, validated at 0.1206 — below the 0.18 leader cluster.

What's next (the real prize)

Our proof corrected only 3 projects. The final ranking (worth $5,000) is judged on HIDDEN projects we can't see. So next we run the AI juror over all 83 projects automatically — so the same correction applies everywhere, not just the 3 we checked by hand.

That pipeline is already built and tested. It's waiting on one thing: Amazon approving our access to run the AI (in progress).

The lesson: don't optimize blind.

First understand exactly what's being measured, then find where the error hides, then fix only that. Everything else followed from those three steps.